

Spring Bean Lifecycle — Structured Theory

Understanding the Bean Lifecycle from the IoC Container's point of view.

1. What is Bean Lifecycle?

The **Bean Lifecycle** is the sequence of stages through which a Spring Bean passes from the time the **IoC Container creates it** until the time the **IoC Container destroys it**.

```
Spring Application Starts
↓
IoC Container starts
↓
Bean Definition is discovered
↓
Bean Object is created
↓
Dependencies are injected
↓
Aware / BeanPostProcessor processing
↓
Initialization
↓
Bean is ready
↓
Bean is used by application
↓
Application shuts down
↓
Destruction
↓
Bean is removed
```

IoC Container controls the lifecycle of Spring Beans.

2. Phase 1 — Bean Definition

Before Spring creates an object, it needs to know: **What Beans exist and how should they be created?** Spring gets this information from **@Component**, **@Service**, **@Repository**, **@Controller**, and **@Bean**.

```
@Component
public class PaymentService {

}
```

Spring's component scanning finds `PaymentService` and creates a **Bean Definition** for it. A Bean Definition is metadata/instructions about a Bean. It can contain the Bean's class, name, scope, dependencies, initialization information, and destruction information.

How IoC manages it: The IoC Container maintains these Bean Definitions so it knows how to create and manage each Bean.

3. Phase 2 — Bean Creation / Instantiation

Spring creates the actual Java object. Conceptually, this is similar to:

```
PaymentService paymentService = new PaymentService();
```

The difference is that Spring performs this creation and manages the resulting object. This object becomes a Spring Bean.

```
Bean Definition
↓
IoC Container
↓
Creates object
↓
```

PaymentService Bean

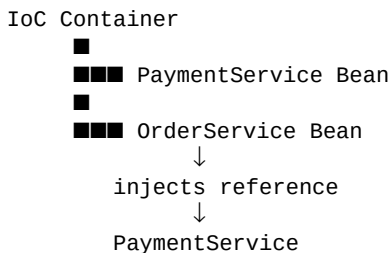
How IoC manages it: The IoC Container decides when the Bean should be created, which class should be instantiated, which scope applies, and how the object should be managed. For a normal singleton Bean, Spring usually creates it when the application context starts.

4. Phase 3 — Dependency Injection

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

After the Bean is created, Spring provides its required dependencies. Spring creates or obtains the PaymentService Bean and injects its reference into OrderService.



OrderService does not have to create the dependency itself. The IoC Container controls where the dependency comes from and provides it to the Bean. This is **Dependency Injection (DI)**.

5. Phase 4 — Aware Interfaces

Spring can provide certain information about the Bean or its environment through special **Aware** interfaces, such as Bean name, ApplicationContext, or BeanFactory.

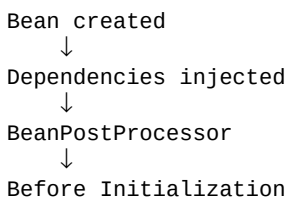
```
@Component
public class MyService implements BeanNameAware {

    @Override
    public void setBeanName(String name) {
        System.out.println(name);
    }
}
```

Spring calls setBeanName(...) and gives the Bean its Spring-managed Bean name. The IoC Container recognizes these interfaces and supplies the requested information.

6. Phase 5 — BeanPostProcessor Before Initialization

A **BeanPostProcessor** allows Spring to perform additional processing on a Bean before its initialization method runs.



Easy meaning: Spring gets a chance to inspect/process the Bean before initialization is completed.

7. Phase 6 — Initialization

The Bean's initialization logic runs after the Bean is created and its dependencies are injected. A common initialization callback is **@PostConstruct**.

```

@PostConstruct
public void init() {
    System.out.println("Bean initialized");
}

```

```

Create Bean
↓
Inject dependencies
↓
@PostConstruct
↓
Bean initialized

```

Initialization is useful when something needs to happen after dependencies are available, such as preparing required data or state.

8. Phase 7 — BeanPostProcessor After Initialization

After initialization, Spring's BeanPostProcessor gets another opportunity to process the Bean using **postProcessAfterInitialization()**.

```

Bean Creation
↓
Dependency Injection
↓
Before Initialization
↓
Initialization
↓
After Initialization
↓
Bean Ready

```

This stage is important because Spring uses Bean post-processing for many framework features.

9. Phase 8 — Bean is Ready

At this point, the Bean has completed its lifecycle initialization and is ready for the application to use. For the default **singleton scope**, Spring normally keeps the same Bean instance inside the IoC Container.

```

IoC Container
↓
OrderService Bean
↓
Application uses it
↓
OrderService Bean

```

10. Phase 9 — Bean Destruction

When the Spring ApplicationContext shuts down, Spring performs the destruction phase for Beans that have destruction callbacks. A common callback is **@PreDestroy**.

```

@PreDestroy
public void destroy() {
    System.out.println("PaymentService destroyed");
}

```

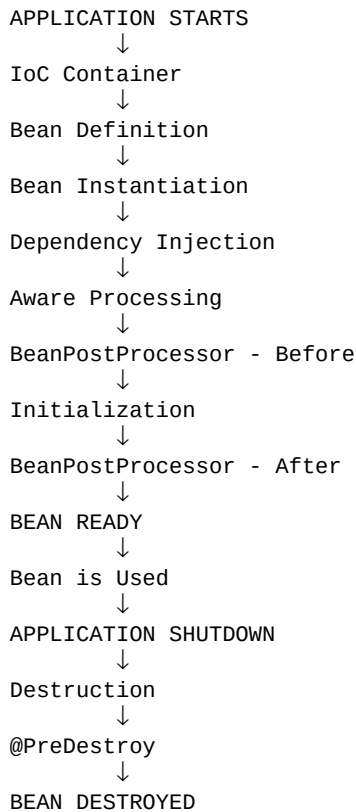
```

Application running
↓
Application shutdown
↓
Spring closes IoC Container
↓
@PreDestroy
↓
Bean cleanup

```

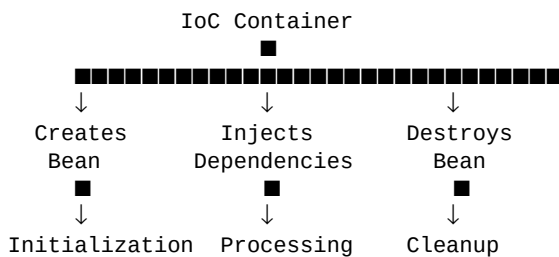
Destruction callbacks are useful for cleanup activities such as releasing resources.

Complete Bean Lifecycle



How IoC Controls the Entire Lifecycle

The IoC Container acts like the **manager** of the Bean.



Without IoC, the programmer creates objects, creates and connects dependencies, initializes, uses, and cleans them. With IoC, the Spring container takes responsibility for these tasks. The control over object creation and management moves from the programmer to the Spring IoC Container.

Example Using Your Project

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        getOrderDetails();
        System.out.println("Order placed");
    }
}

@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment done");
    }
}
```

```

    }
}
SpringApplication.run()
↓
Spring IoC Container starts
↓
Component Scan
↓
Finds @Component
↓
Finds OrderService
↓
Finds PaymentService
↓
Creates Beans
↓
Injects PaymentService into OrderService
↓
Initializes Beans
↓
Beans are ready
↓
context.getBean(OrderService.class)
↓
OrderService reference obtained
↓
order.placeOrder()
↓
Payment done
Order Details
Order placed
↓
Application stops
↓
Beans are destroyed

```

Bean Lifecycle vs Bean Scope

Bean Lifecycle answers: What happens to the Bean from creation to destruction?

Create → Inject → Initialize → Use → Destroy

Bean Scope answers: How many instances of the Bean are created and what scope they are associated with?

Singleton → one Bean instance per IoC container
 Prototype → new instance when requested
 Request → associated with an HTTP request
 Session → associated with an HTTP session

Bean Scope = How many / what scope? Bean Lifecycle = What happens to it?

Final Exam-Ready Definition

The Spring Bean Lifecycle is the sequence of stages through which a Bean passes under the control of the Spring IoC Container. The process begins with Bean definition and instantiation, followed by dependency injection, aware processing, pre-initialization processing, initialization, post-initialization processing, and making the Bean available for use. When the application context is closed, Spring performs the Bean's destruction callbacks and releases the managed Bean. The IoC Container is responsible for controlling these stages, which is a practical example of Inversion of Control.

Easy Memory Line

```

Definition
↓
Creation
↓
Dependency Injection
↓

```

Aware
↓
Before Initialization
↓
Initialization
↓
After Initialization
↓
Ready / Use
↓
Destroy

Shortest way to remember: Spring finds → creates → injects → initializes → uses → destroys the Bean.